COMP1322 Programming II
Coursework

Module Code:	COMP1322		
Module Title:	Programming II		
Module Leader:	Asst. Prof. Hasmath Farhana TA		
Assessment Type:	Individual Coursework	**Weightage:**	30%
Submission Due Date:	27 April 2026, 9am (Malaysia time)		
Method of Submission:	Online submission via the Blackboard		
GenAI Tier Specification	**Tier 1: No GenAI use [applies All - Entire work]**		

This assessment relates to the following Module Learning Outcomes:

B. Subject Specific Intellectual and Research skill	B2 Construct Java applications with Graphical User Interfaces B3 Construct multi-threaded Java applications B4 Use persistent storage for Java applications B5 Perform testing on Java programs using JUnit

©2025/26 University of Southampton

Coursework Specification:

Issue Date: 11 March 2026
Submission Deadline: 27 April 2026, 9am (Malaysia time)

1. General Notes

- This is an **individual coursework**.
- This coursework contributes **30%** to the overall module mark.
- Your code should be clean and easy to read and understand. You should minimize unnecessary redundancies.
- Include comments accordingly, to explain important parts of your program.
- This coursework is meant to see your **creativity** (GUI Design) and the **functionality**.
- Submit ONLY to the **Blackboard submission link.**

2. Project Description

This is a major piece of coursework worth **30%** and will put into practice all the skills you have developed in the module so far into building a full working application. It is expected that you will spend around 40 hours on this coursework. This coursework builds on everything you have learnt (and will learn), especially:

- **JavaFX**, including controls, graphics, event listeners
- **File Input and Output**
- **Error handling**
- **Assertion (testing)**
- In addition, you need to consider the good software design practices you've learnt in Programming. Refer here.
- **Advanced features** (synchronization, recursion, multi-threading), may or may not be necessary, depending on your application type.

As a professional system developer, you have recently received a request to develop a **management system** for a newly opened business. This business can be of any types, such as
(**CHOOSE ONE TYPE**):

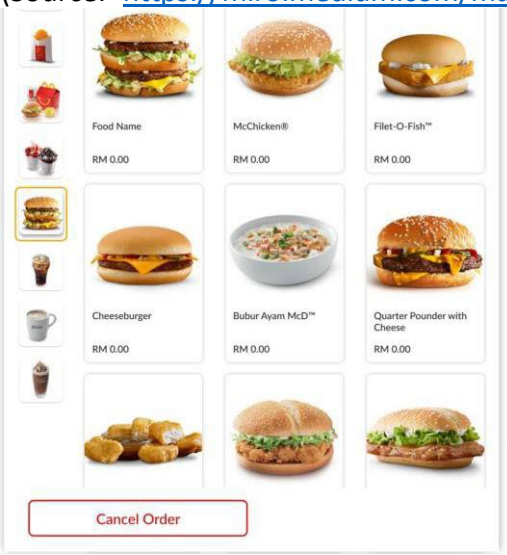
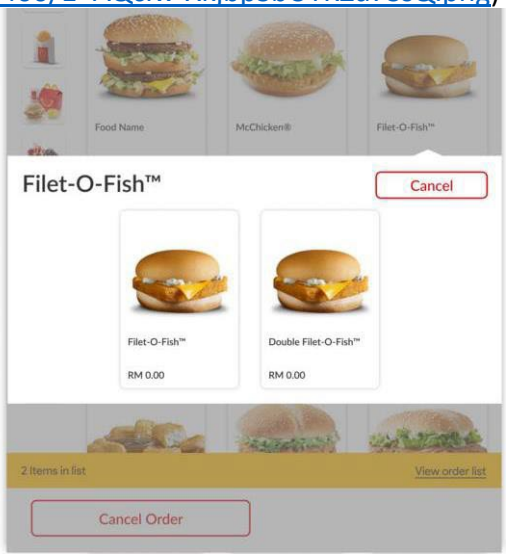
- **Restaurant/café** (Restaurant point of sales, such as McDonalds)
- **Hotel** (Hotel management system)
- **Retail** (such as 7Eleven, Speedmart)
- **Clinic** (Clinic management system)
- **Academic** (Managing student grades, lecturers, etc.)
- It could be other things; you may decide as long as you cover the marking criteria explained below.

Create the **management system application** using **Java and JavaFX library** for the user interface using the program features and marking criteria explained in Section 3 below.

©2025/26 University of Southampton

3. Program Features and Marking Criteria

This coursework will be marked using the following criteria:

Category	Mark Allocation
Software Features	**Total 40**
A. Login/Privilege 1. Your app has login functionality. 2. Your app needs to have at least two privileges, user and admin. 3. User is for normal users, such as customers who want to order at McDonalds. 4. Admin is for the business owner, with features such as reporting, adjusting stocks, etc.	8
B. File Input and Output, including but not limited to: 1. Your app needs to have data storage, such as text or binary file, to keep data such as user inputs, transactions, etc. 2. Extra points for implementing database such as MySQL or SQLite.	8
C. CRUD for Customer Related Data (for Normal User) 1. **C**reate: We can make a new order/purchase in the app. 2. **R**ead: Data can be viewed in a form of tables, text, etc. 3. **U**pdate: Users can update the existing order/purchase. 4. **D**elete: Data can be deleted. Refer to this CRUD example: (Source: https://miro.medium.com/max/1400/1*AQcIw-XkjbpSbUYKZd7SsQ.png)  	8
D. CRUD for Master Data (for Admin) 1. **C**reate: Data can be entered to the system, such as inputting new food items, etc. 2. **R**ead: Data can be viewed in a form of tables, text, etc.	8

©2025/26 University of Southampton

3. **U**pdate: Data can be updated, such as stock adjustments, item name and category adjustments, etc.
4. **D**elete: Data can be deleted.

Refer to this CRUD example:
(Source: https://www.pixeltrice.com/spring-boot-elasticsearch-crud-example/)

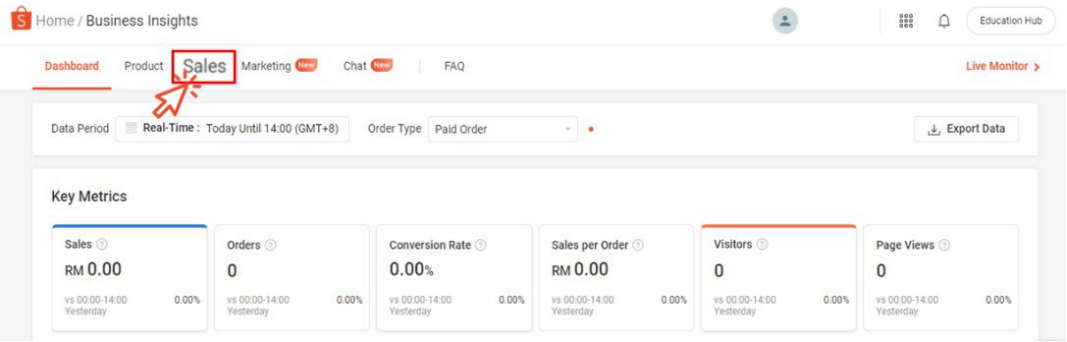
Add Product | Go Home

Id	Name	Description	Price	Action
1001	Book	Great book to read	101.5	Update Delete
1002	Computer	Wonderful product	201.8	Update Delete
1003	Camera	Nice to have one.	501.8	Update Delete
1005	NoteBook	To document something.	20.98	Update Delete
1006	Car	Super Easy to Drive	1001.69	Update Delete

E. Report Feature, examples: 8

1. Invoice/bill page that will show the recap of a purchase.
2. Sales by category, date, customer location, etc. (average, sum, maximum, minimum).
3. Report can be filtered, such as there will be combo box to select what report to view, or text box to filter by minimum to maximum range.

Refer to the following report example:
(Source: https://seller.shopee.com.my/edu/article/850)



(Source: https://www.forcemanager.com/blog/how-to-make-a-sales-report/)

Weekly Revenue Sales Report ($1000)							
FORCEMANAGER	Monday	Tuesday	Wednesday	Thursday	Friday	Total	Forecasted
Sales rep #1	2	1.2	5	0.7	0.3	9.2	10
Sales rep #2	5	2	1.8	0	0	8.8	10
Sales rep #3	10	2	1	1	4	18	25
Sales rep #4	0.7	0.5	4	3	6	14.2	20
Sales rep #5	0	5	8	2	3	18	15
Sales rep #6	0	6	6	0	0.9	12.9	15

©2025/26 University of Southampton

User Interface (JavaFX)	Total 15
F. Consistency, including but not limited to: 1. Verify that the font size/type is the same throughout the application. 2. Verify that the font colour is the same on all stages/windows/forms.	3
G. Functionality/Practicality, including but not limited to: 1. Verify that the correct use of UI elements (e.g., button, text field, password field, text area, checkbox, radio button, etc.) for the user inputs. 2. Verify that the UI elements are working after user interactions.	4
H. Validation, including but not limited to: 1. Verify if the validation error is displayed correctly when a user enters invalid data. 2. Verify that there are no spelling mistakes or errors on the entire application.	4
I. Layout Design, including but not limited to: 1. Verify that all UI elements are correctly aligned. 2. Verify that all UI elements are positioned in convenient places. 3. Verify if the padding, border, and margins have proper spacing.	4
Software Design and Testing	**Total 25**
J. Flexibility/Scalability, including but not limited to: 1. Highly readable code, such as appropriate variable/method naming, and put comments on the important parts of the code. 2. Usage of arrays is preferred over multiple variables. 3. Usage of loops is preferred over lengthy IF-else. 4. Using static HashMap or ArrayList to keep important settings.	4
K. Efficiency, including but not limited to: 1. Code with high cohesion: Each method/class should be responsible for one logical task. 2. Code with loose coupling: Different classes or methods should not be very dependent on each other. 3. Responsibility-driven design: Each class should be responsible for manipulating its own data. 4. Minimize the number of loops as much as possible.	7
L. Error handling and prevention, including but not limited to: 1. Exception handling (try-catch). 2. Prevent NullPointerException. 3. Prevent various errors that could happen, such as division by zero, out of bounds, etc. 4. Encapsulation strategies: Getters and setters, private or protected, and decision on when to create classes. 5. Think carefully about making loops and conditions, whether they are robust enough for different kinds of inputs.	4

©2025/26 University of Southampton

M. Testing (Assertions), including but not limited to: 1. Use Junit testing (assertions) for various methods in the code, especially the complicated ones.	10
Demonstration & Explanation You DO NOT need to create a report. Just the project. However, you will need to demonstrate the system you have developed during the lab sessions (Week 12).	**Total 20**
N. Live Demonstration, including but not limited to: 1. Successfully run the application and navigate through key features. 2. Demonstrate all required functionalities, including login, CRUD operations, and reporting. 3. Showcase UI interaction, validation, and real-time updates (if any). 4. Ensure no unexpected crashes or errors occur during the demonstration.	10
O. Code Walkthrough & Explanation, including but not limited to: 1. Explain main components of the code, such as class structures, data handling, and event listeners. 2. Justify choices related to data storage (MySQL/SQLite), multi-threading, and UI design. 3. Describe error handling strategies and demonstrate exception handling techniques. 4. Ensure clarity, conciseness, and structured explanation without reading from a script.	5
P. Q&A and Debugging Challenge, including but not limited to: 1. Answer technical questions related to data structures, OOP principles, and efficiency. 2. Identify and resolve a small bug or modification given by the lecturer during the presentation. 3. Justify why a certain approach/technique/design was chosen. 4. Demonstrate the ability to think critically and adjust code on the spot if needed.	5

4. Submission Instructions

You need to submit the **whole project folder, zipped**. Any dependencies, such as images, audio, video, database (MySQL or SQLite), any storage files, should be zipped together with the project. You must include a readme.txt if there are some special ways to open your project.

Mandatory Presentation (teaching week 12) – No submission will be marked without the demo. Presentation schedule will be released by teaching Week 10.

©2025/26 University of Southampton

Note:
While you are encouraged to implement the concepts introduced in lectures and labs, you are **not restricted** to only using those techniques. **Software development is a continuous learning process**, and a skilled programmer must be able to **identify and integrate new concepts** as needed.

During your project development, you may **encounter scenarios that require solutions beyond the covered material**. In such cases, you are encouraged to **research, explore, and apply new ideas** that best fit your application's requirements. This approach mirrors **real-world programming**, where adaptability and **continuous learning** are key to solving complex problems.

Embracing new knowledge, when necessary, is a hallmark of a great programmer. **If your application demands a feature or technique not discussed in class, do not hesitate to explore it.** Innovation and curiosity will help you grow **beyond the boundaries of this coursework**—and that is the **ultimate objective of this coursework.**

Good luck, we wish to see your wonderful creations!

Any work submitted after the deadline's time will be subject to the standard University late penalties unless an extension has been granted, in writing by the Senior Tutor, in advance of the deadline. Details on the University's late penalties can be found here:
• https://www.southampton.ac.uk/~assets/doc/quality-handbook/Late%20Submission.pdf

©2025/26 University of Southampton